

Linguagem de Programação: Python

Antonio Mauro Rezende, Ph.D.
Biotecnologia Aplicada ao Estudo de Patógenos
Instituto René Rachou, Fiocruz Minas



Disclaimers:

- O conteúdo dessa apresentação representa a minha opinião. Ele não representa o posicionamento das instituições citadas ao longo da minha fala.
- Apresentação foi criada com o suporte de ferramentas de IA, e eu me responsabilizo por todo conteúdo.



- Lançada em 1991 por Guido van Rossum – Holanda
- Interpretada – códigos executados pelo interpretador Python
- Atualmente estamos na versão 3
- Muito popular
- Milhares de bibliotecas para bioinformática
- Aplicações em: ciência de dados, IA, web, sistemas embarcados,



Donate



Search

GO

Socialize

About

Downloads

Documentation

Community

Success Stories

News

Events

```
# Simple output (with Unicode)
>>> print("Hello, I'm Python!")
Hello, I'm Python!

# Input, assignment
>>> name = input('What is your name?\n')
>>> print(f'Hi, {name}.')
What is your name?
Python
Hi, Python.
```

Quick & Easy to Learn

Experienced programmers in any other language can pick up Python very quickly, and beginners find the clean syntax and indentation structure easy to learn. [Whet your appetite](#) with our Python 3 overview.

1 2 3 4 5

Python is a programming language that lets you work quickly and integrate systems more effectively. [>>> Learn More](#)

Get Started

Whether you're new to programming or an experienced developer, it's easy to learn and use Python.

[Start with our Beginner's Guide](#)

Download

Python source code and installers are available for download for all versions!

Latest: [Python 3.14.7](#)

Docs

Documentation for Python's standard library, along with tutorials and guides, are available online.

docs.python.org

Jobs

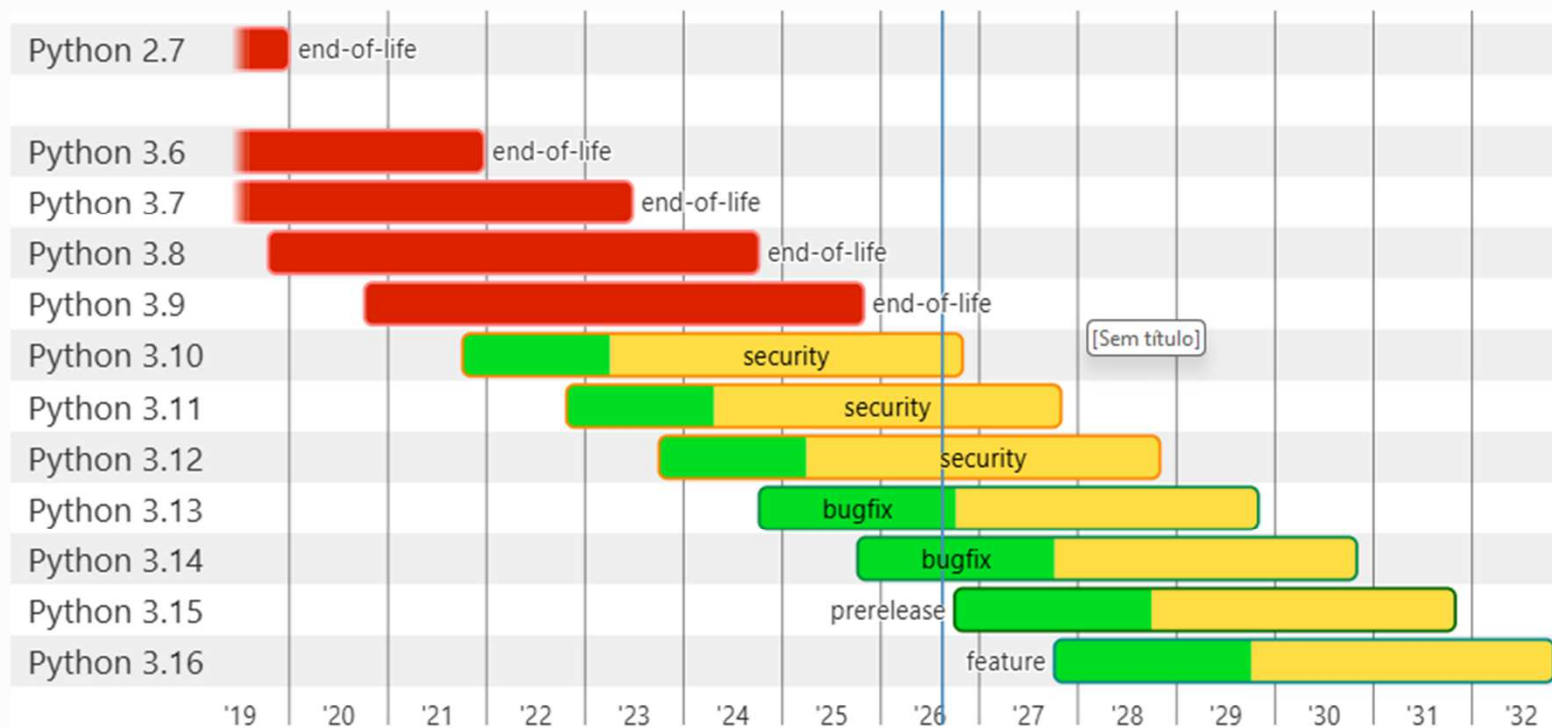
Looking for work or have a Python related position that you're trying to hire for? Our **relaunched community-run job board** is the place to go.

jobs.python.org



Active Python releases

For more information visit the [Python Developer's Guide](#).



Fonte: <https://www.python.org/downloads/>

Iniciar o uso:

- Python

```
(base) ARezende@rezende:~$ python
Python 3.11.4 (main, Jul 5 2023, 13:45:01) [GCC 11.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import sys
>>> sys.version
'3.11.4 (main, Jul 5 2023, 13:45:01) [GCC 11.2.0]'
>>> █
```

- “>>>” significa que o interpretador está esperando a instrução
- `1+1`
- `r = 5`
- `3.14*(r**2)`
- `print (“Bem vindo ao Python”)`
- Esse modo é conhecido como shell: aprendizado e experimentar



Iniciar o uso:

- Modo Script:
 - Você edita um programa (algoritmo) e salva em um arquivo
 - Nesse modo, se quiser exibir algo precisar utilizar a função “print”
 - Dentro de um programa (script), as expressões que começam com “#” são comentários

```
###Programa para somar dois numeros inteiros
print("digite o primeiro numero")
a = input()
print("digite o segundo numero")
b = input()
print("soma:",int(a) + int(b))
```





- Atribuição de variáveis “=”

```
>>> idade = 18
>>> x = 2 * 3
>>> y = idade + x
>>> y = y + 1
```

- Atribuição composta = modifica o valor da variável através de uma operação matemática

atribuição composta	é equivalente a
<code>x += 3</code>	<code>x = x + 3</code>
<code>x += y * 2</code>	<code>x = x + y * 2</code>
<code>preco -= desconto</code>	<code>preco = preco - desconto</code>
<code>num /= 2</code>	<code>num = num / 2</code>
<code>a *= b + 1</code>	<code>a = a * (b + 1)</code>



- Nome da variável
 - Letras (A-Z, a-z), dígitos (0-9) e underscore(_)
 - Não pode começar com um dígito

```
(base) ARezende@rezende:~/aula_2_programacao$ python
Python 3.11.4 (main, Jul 5 2023, 13:45:01) [GCC 11.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> 9teste = "oi"
      File "<stdin>", line 1
        9teste = "oi"
        ^
SyntaxError: invalid decimal literal
>>> teste = "oi"
>>>
```

Iniciar o uso:

- Idade, idade, IDADE são diferentes
- Algumas palavras do ambiente não podem ser utilizadas. Ex.: if, for, while



- int
- float
- bool
- str

```
>>> x = 1
>>> type(x)
>>> y = 3.0
>>> type(y)
```

- int(valor)
- float(valor)
- bool(valor)
- str(valor)

```
x = "10"
y = 2
z = int(x) - y
```



- int
- float
- bool
- str

```
>>> x = 1
>>> type(x)
>>> y = 3.0
>>> type(y)
```

- int(valor)
- float(valor)
- bool(valor)
- str(valor)

```
x = "10"
y = 2
z = int(x) - y
```

Na conversão de float para int as casas decimais são descartadas

```
(base) ARezende@rezende:~/aula_2_programacao$ python
Python 3.11.4 (main, Jul 5 2023, 13:45:01) [GCC 11.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> x = 1.99
>>> int(x)
1
```



Símbolo	significado
+	soma
-	subtração
*	multiplicação
/	divisão
//	divisão inteira*
%	resto da divisão
**	potenciação

Funções matemáticas: precisa de

from math import *

Funções trigonométricas: **sin(x)**, **cos(x)**, **tan(x)**

Valor absoluto: **fabs(x)**. Ex.: **fabs(-11) ==> 11**

Raiz quadrada: **sqrt(x)**. Ex.: **sqrt(9) ==> 3**.

Logaritmos: **log(x)** (logaritmo natural, base e), **log10(x)** (base 10)

Arredondamento: **ceil(x)** (arredonda pra cima), **floor(x)** (arredonda pra baixo)

Constantes: **pi**, **e**

* A **divisão inteira** retorna apenas a parte inteira do resultado. Exemplos:

- Divisão normal: **10 / 4 ==> 2.5**
- Divisão inteira: **10 // 4 ==> 2**

Fonte: <https://rodrigorgs.github.io/aulas/ilp/01b-expressoes#3>

Atento com o operador “+” com strings

```
nome = "Fulano"
sobrenome = "de Tal"
nome_completo = nome + " " + sobrenome
```



- Não é possível concatenar string com um número:

```
ano = 2001
x = 'maio de ' + ano
# TypeError: can only concatenate str (not "int") to str
```

- Para concatenar é preciso converter para string:

```
ano = 2001
x = 'maio de ' + str(ano) 👁👁
```

interpolação de strings

```
nome = 'Fulana'
idade = 18
frase = f'{nome} tem {idade} anos' 👁👁
```

```
total = 1 / 7 👁👁
frase = f'Resultado: {total:.2f}' 👁👁
```

Fonte: <https://rodrigorgs.github.io/aulas/ilp/01b-expressoes#10>



- Escrever na tela: `print()`

```
a = 1
b = 2
c = 3
print(f'{a} {b} {c}') 🐍
```

Se você quiser evitar a quebra de linha, use `end=""`. Exemplo:

```
print("Oi, ", end="")
print("pessoal!")
```

```
ddd = 71
telefone = "5555-5555"
print("(", ddd, ") ", telefone, sep="")
```



- Ler do teclado `input()` – sempre retorna string

```
print("Qual é o seu nome?")  
nome = input()  
print("Oi", nome)
```

```
print("Digite o primeiro número:")  
a = input()  
print("Digite o segundo número:")  
b = input()  
print("Soma:", a + b)
```

Qual o problema desse programa?

```
a, b, c = input().split()  
a = int(a)  
b = int(b)  
c = int(c)  
print(a + b + c)
```

Qual o comportamento com essa forma de entrada de dados?



- If/else/elif

```
idade = int(input("Digite sua idade: "))  
if idade < 18:  
    print("Você não é adulto.")  
    print("Favor trazer autorização do responsável.")  
print("Programa finalizado.")
```

- Importante destacar a **IDENTAÇÃO**

- > - é maior que
- < - é menor que
- >= - é maior ou igual a (não escrever =>)
- <= - é menor ou igual a (não escrever =<)
- == - é igual a (não confundir com =)
- != - é diferente de

Operadores relacionais



```
preco = float(input())
quantidade = int(input())

total = preco * quantidade
if quantidade > 10:
    total *= 0.95

print(total)
```



```
idade = int(input("Digite sua idade: "))
if idade >= 18:
    print("Você é adulto.")
else:
    print("Você não é adulto.")
print("Fim")
```

Testes aninhados

```
idade = int(input("Digite sua idade: "))
if idade >= 18:
    print("Você é adulto.")
else:
    if idade >= 12:
        print("Você é adolescente.")
    else:
        print("Você é criança.")
print("Fim")
```



```
nota = float(input())
if nota > 9.0:
    print("Excelente!")
elif nota > 7.0:
    print("Muito bom!")
elif nota > 5.0:
    print("Razoável!")
else:
    print("Estude mais!")
```



- Operadores lógicos
 - not
 - and
 - or
 - Ordem de precedência not, and, or

```
x = int(input())  
if not (x <= 0) and x % 2 == 1:  
    print("Positivo impar")
```

```
x, y = input().split()  
x = int(x)  
y = int(y)  
if y == 1 or x > 0 and x < 5:  
    print("Ok")
```



- Repetição

```
print("Quantas pessoas?")  
n = int(input())
```

```
for i in range(n):  
    print("Qual seu nome?")  
    nome = input()  
    print("Olá,", nome)
```

`range(n)` gera a sequência de números 0, 1, 2, ..., n - 1

A cada repetição, a **variável de controle** assume o próximo valor da **sequência**

```
for i in range(5):  
    bloco de instruções...
```

range(início, fim, passo) (passo é opcional; se não for informado, é 1)



- Repetição

```
i = 1
while i <= 3:
    print(i)
    i = i + 1
print("Fim")
```

```
# escrever a tabuada de 1 até a tabuada de 5
for i in range(1, 6):
    // escrever a tabuada de i
    for j in range(1, 10):
        print(i, "x", j, "=", (i * j))
```



- **Funções** são blocos de código que podem ser **chamados** em outros pontos do código
 - **Chamar** uma função significa pedir **executar** o seu bloco de código
- Python fornece várias funções prontas para serem usadas: `print()`, `input()`, `sqrt()`, `int()`



```
def desenha(largura, altura, simbolo):  
    l = largura  
    a = altura  
    c = simbolo  
  
    print(c*l)  
    for i in range(a-2):  
        print(c + (" " * (l-2)) + c)  
    print(c * l)  
  
print("Digite o valor da largura do retangulo")  
x = int(input())  
print("Digite o valor da altura do retangulo")  
y = int(input())  
  
desenha(x, y, '*')
```

Variáveis locais: l, a, c

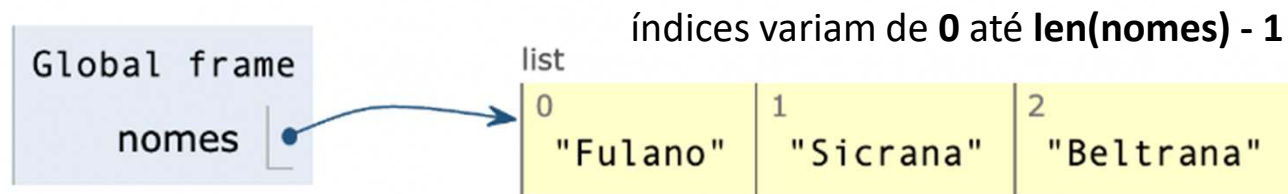
Variáveis globais: x, y



- Listas

Uma lista é representada por colchetes contendo elementos separados por vírgulas. Exemplos:

```
vazia = []  
notas = [9.8, 7.5, 8.3]  
nomes = ["Fulano", "Sicrana", "Beltrana"]  
x = [notas, nomes]  
usuarios = [{"usuario", "senha"}, {"admin", "123"}]
```



Use a função **len(x)** para obter o comprimento de uma lista x

```
numeros = [2, 2, 2, 2]  
print(len(numeros))
```



Se você souber o número de elementos de uma lista, você também pode usar uma atribuição com diversas variáveis. Exemplo:

```
lista = [2, 4, 6]  
x, y, z = lista
```

Se o número de variáveis do lado esquerdo do símbolo = não coincidir com o número de elementos da lista, você verá um dos seguintes erros:

- `ValueError: too many values to unpack`
- `ValueError: not enough values to unpack`



```
L1 = [10, 20]
```

```
L2 = [30, 40]
```

```
L2.append(50) # Insere 50 no final da lista
```

```
del L2[2]      # Remove elemento no índice 2
```

```
L2.insert(1, -10) # Insere -10 no índice 1, empurrando os elementos para a frente
```

```
del L2[1]      # Remove elemento no índice 1, puxando os próximos para frente
```

```
L3 = L1 + L2   # Cria uma nova lista com os elementos de L1 e L2
```

- **clear**
- **extend**
- **reverse**
- **pop**
- **remove**
- **join**



```
# Transformando elementos
```

```
lista = [1, 2, 3, 4, 5]
```

```
l2 = [n * 2 for n in lista]
```

```
entrada = ["4", "8", "3"]
```

```
l3 = [int(n) for n in entrada]
```

```
# Filtrando elementos
```

```
lista = [-1, -4, 2, -3, 4]
```

```
positivos = [n for n in lista if n > 0]
```

```
# Filtrando e transformando
```

```
pares = [n * 10 for n in lista if n % 2 == 0]
```



Dicionário

```
notas = {'joao': 8,  
        'maria': 7,  
        'jose': 10 }  
print(notas)
```

```
>>> {'joao': 8, 'maria': 7, 'jose': 10}
```

- Um dicionário é acessado por suas **chaves**.
- Diferentemente de listas, onde o índice é um número, dicionários utilizam suas chaves como índice.
- Observe o exemplo abaixo:

```
notas = {'joao': 8, 'maria': 7, 'jose': 10}  
print("Nota de João: ", notas['joao'])  
print("Nota de Maria: ", notas['maria'])  
print("Nota de José: ", notas['jose'])  
>>> Nota de João: 8  
>>> Nota de Maria: 7  
>>> Nota de José: 10
```



Dicionário

- Podemos obter uma lista com as chaves ou com os valores do dicionário.
- Observe o exemplo abaixo:

```
notas = {'joao': 8, 'maria': 7, 'jose': 10}
print(notas.keys())
>>> dict_keys(['joao', 'maria', 'jose'])

print(notas.values())
>>> dict_values([8, 7, 10])
```

- Em Python, podemos ter dicionários nos quais as chaves são associadas a listas ou mesmo a outros dicionários.
- Imagine uma relação de estoque de mercadorias onde teríamos, além do preço, a quantidade em estoque.
- Observe o exemplo abaixo:

```
estoque = {
    'cerveja': [350, 12.50],
    'agua': [125, 4.00],
    'refrigerante': [235, 6.30]
}
```



Matrizes

- A linguagem Python **não possui*** o conceito de **matriz**, mas podemos emular essa estrutura criando uma **lista contendo várias listas de mesmo tamanho**.
- Cada lista interna representa uma **linha** da matriz, como veremos a seguir

```
m = [  
    [2, 3, -1, 5],  
    [1, -2, 3, 4],  
    [4, -5, 6, 7]  
]  
print(m[2][1]) # Saída: -5
```

índices	0	1	2	3
0	2	3	-1	5
1	1	-2	3	4
2	4	-5	6	7



Matrizes

5	3	4
6	0	2
1	9	8

=

0	1	2	3	4	5	6	8	9
---	---	---	---	---	---	---	---	---

5	3	4
6	4	2
1	9	8

O filtro da mediana pode ser usado para remover ruídos ou suavizar imagens.





Matrizes

```
pixels = [[5, 3, 4],  
          [6, 1, 2],  
          [1, 9, 8]]
```

```
lista_valores = []  
for l in range(3):  
    for c in range(3):  
        lista_valores.append(pixels[l][c])
```

```
lista_valores.sort()  
pixels[1][1] = lista_valores[4]  
print(pixels)
```

5	3	4
6	1	2
1	9	8



5	3	4
6	4	2
1	9	8



Arquivos

- **Ler linha a linha**
- **Carregar todo na memória**

with open(nome, modo) as nome:

- ``with`` = context manager
- ``nome`` é o arquivo a ser aberto (pode ser uma variável ou uma string contendo o caminho);
- ``modo`` é o modo de abertura:
 - ``r``: somente leitura (padrão).
 - ``w``: escrita (sobrescreve o arquivo)
 - ``a``: **append**, ou escrever a partir do final
 - ``b``: binário (opcional; o padrão é ``t`` para texto).
 - ``+``: leitura e escrita

Importante: todo o código dentro do ``with`` precisa estar indentado!



Arquivos - Leitura

```
arquivo_fasta = 'seqs.fasta'

# leitura linha a linha
# retorna strings
✓ with open(arquivo_fasta, 'r') as f:
✓     for linha in f: # Iterar sobre f: cada iteração retorna uma linha como string (com o \n no final).
        linha = linha.strip() # remove espaços em branco nas extremidades
        print(linha)
```

```
# leitura do arquivo de uma vez só
# retorna uma lista de strings
with open('seqs.fasta', 'r') as f:
    linhas = f.readlines()
    print(linhas)

for linha in linhas:
    print(linha)
```



Arquivos - Escrita

```
arquivo_saida = 'saida.fasta'

# se o arquivo ainda não existir, ele será criado
# w: escrita sobrescrevendo
# a: escrita no final (não sobrescreve)

# with open(arquivo_saida, 'w') as f:
#     f.write('a')

with open(arquivo_saida, 'a') as f:
    f.write('\nATCGATCG')
```

Exemplos de Código: Calcular conteúdo GC

```
# Definindo sequências de DNA
seq_A = "ATGATCTCGTAATTAACCGGAATTTTGGGCC"
seq_B = "GGCCTTAAGTTTAACCCGGAATTTAAAGGCCCAAA"

def calcular_conteudo_gc(sequencia):
    """
    Retorna o percentual de bases G ou C em 'sequencia'.
    """
    # Verifica se a sequência não está vazia

    if not sequencia:
        return 0.0

    # Conta cada base G e C
    cont_G = sequencia.count('G')
    cont_C = sequencia.count('C')

    # Soma G + C e obtém o comprimento total
    total_GC = cont_G + cont_C
    tamanho_total = len(sequencia)

    # Calcula percentual
    percentual_gc = (total_GC / tamanho_total) * 100
    return percentual_gc

# Aplica a função e imprime
gc_A = calcular_conteudo_gc(seq_A)
gc_B = calcular_conteudo_gc(seq_B)

print(f"Conteúdo GC da Sequência A: {gc_A:.2f}%")
print(f"Conteúdo GC da Sequência B: {gc_B:.2f}%")
```

Exemplos de Código: Tamanho médio seq

```
# Carregar as três sequências em um dicionário para facilitar a iteração
seq_A = ("LRSSSQNSSDKPVAHVVAHQVEEQLEWLSQRANALLANGMDLKNQLVVPADGLY"
         "LVYSQVLFKGQGPCDYVLLTHTVSLRSSSDK")
seq_B = ("KPA AHLIGDPSKQNSLLWRANTDRAFLQDGFSLSNNSLLVPTSGIYFVYSQVVFSG"
         "KAYSPKATSSPLYLAHEVQLFSS")
seq_C = ("CPQGKYIHPQNSICCTKCHKGTLYNDCPGPGQDTCRECESGSFTASENHLRHCL"
         "SCSKCRKEMGQVEISSCTVDRDTCGCR")
```

```
lista = {
    "Sequência A": seq_A,
    "Sequência B": seq_B,
    "Sequência C": seq_C
}
```

```
# Imprimir sequências com 80 ou mais aminoácidos
print("Sequências com >= 80 aminoácidos:")
```

```
for nome, seq in lista.items():
    if len(seq) >= 80:
        print(f"    • {nome}: {len(seq)} AAs")
```

```
# Usa-se len() para medir o comprimento e filtramos as que têm >= 80
```

```
# Imprimir sequências maiores que a média de tamanho
```

```
comprimentos = [len(seq) for seq in lista.values()]
media = sum(comprimentos) / len(comprimentos) # calcular média
```

```
print(f"Média de tamanho: {media:.2f} AAs")
```

```
print("Sequências com comprimento acima da média:")
```

```
for nome, seq in lista.items():
    if len(seq) > media:
        print(f"{nome}: {len(seq)} AAs")
```

Exemplos de Código: Checar Sequencia

```
alfabeto_dna = set("AGTC")           # DNA: A, G, T, C
alfabeto_rna = set("UCAG")          # RNA: U, C, A, G
alfabeto_proteina = set("ACDEFGHIKLMNPQIRSTVWY") # Proteínas: 20 aa padrão

# Lista das sequências a serem analisadas
sequencias = [
    "ATCDLASKWNNHTLCAAHCIARRYRGGYCNKAVCVCRN",
    "TATTAACCGGGTTTAACTAGCATGCATGATTAACCAGTACATCTTTT",
    "ATCBDLASKWNNHTLCAAHCIARRYRGGYCNSJAVCVCRN"
]

print("Verificação de tipo de cada sequência:\n")

for idx, seq in enumerate(sequencias, start=1):
    # Padroniza para maiúsculas e converte em conjunto
    seq = seq.upper()
    conjunto_seq = set(seq)

    # Testa se todos os caracteres estão dentro de cada alfabeto
    eh_dna = conjunto_seq.issubset(alfabeto_dna)
    eh_rna = conjunto_seq.issubset(alfabeto_rna)
    eh_proteina = conjunto_seq.issubset(alfabeto_proteina)

    # Inicialmente assume que não pertence a nenhum
    tipo = "Nenhuma delas"
    invalidos = set()

    # Define o tipo de sequência com base nos testes acima
    if eh_dna:
        tipo = "DNA"
    elif eh_rna:
        tipo = "RNA"
    elif eh_proteina:
        tipo = "Proteína"
    else:
        # Se não for puro de nenhum alfabeto, identifica os caracteres inválidos
        alfabeto_total = alfabeto_dna.union(alfabeto_rna, alfabeto_proteina)
        invalidos = conjunto_seq.difference(alfabeto_total)

    # Imprime o resultado
    print(f"Sequência {idx}: {seq}")
    print(f"- Tipo detectado: {tipo}")
    if invalidos:
        # Ordena para facilitar leitura
        lista_inv = sorted(invalidos)
        print(f"- Caracteres inválidos: {lista_inv}")
    print() # linha em branco para separar as análises
```

Obrigado!

antonio.rezende@fiocruz.br

